

TP_PrograB

Guía completa de diseño y funcionamiento

Copa Internacional de Clubes

Java 24 · POO · JSON · JDBC · PostgreSQL · JavaFX

Estado documentado: septiembre de 2026

Objetivo: entender el proyecto de punta a punta y poder defender cada decisión.

Este documento describe el código que existe actualmente. No presenta como terminadas las funciones que solo tienen una clase preliminar. Está pensado como material de estudio, documentación técnica y apoyo para la defensa oral.

Contenido

Contenido	2
1. Qué sistema se está construyendo	5
El recorrido principal	5
2. Organización de carpetas	6
Regla de dependencias	6
3. Configuración del proyecto	6
pom.xml	6
4. Entrada JSON y DTO	7
Por qué existen DTO y dominio	7
Jerarquía de DTO	7
Adaptador de CountryDto	7
5. TournamentLoader: del archivo a los objetos	7
Paso 1 - leer y deserializar	7
Paso 2 - catálogo único de países	7
Paso 3 - personas e IDs internos	8
Paso 4 - validación	8
6. Validación de datos	9
Documentos repetidos	9
7. Modelo de personas y países	9
Person	9
Country	9
Player	9
Coach y Referee	9
8. Team y la fuerza competitiva	10
Cálculo actual	10
9. Championship: coordinador del torneo	10
Construcción	10
Dos formas de aleatoriedad	10
10. Sorteo equilibrado de zonas	11
11. Generación de los 24 partidos	11
12. Regla de árbitros	11
13. Match y sus subtipos	12
Estado inicial y estado final	12
Subtipos eliminatorios	12
14. Lineup y FormationType	12
Selección	12

15. MatchSimulator paso a paso	13
1 - impedir doble ejecución	13
2 - decidir tácticas	13
3 - fuerza, localía y sorpresa	13
4 - cantidad de goles	13
5 - incidencias	13
6 - cierre	13
16. Incidencias	14
Orden y almacenamiento	14
17. Salida por terminal	14
Partidas ya terminadas	14
18. Persistencia por serialización	15
Por qué tantas clases implementan Serializable	15
Random y MatchSimulator son transient	15
Decisión del main	15
19. PostgreSQL y JDBC	15
Modelo relacional observado	15
20. DatabaseConnection	16
Responsabilidades	16
21. CityRepository	16
22. StadiumRepository e integración	17
Carga encadenada	17
Qué valida loadVenues	17
23. Clases de prueba	17
24. JavaFX: qué existe y qué falta	18
25. Programación Orientada a Objetos	18
Encapsulamiento	18
Herencia	18
Abstracción	18
Polimorfismo	18
Composición	18
Separación de responsabilidades	18
26. Correspondencia con la consigna	19
27. Riesgos y puntos a mejorar	19
28. Cómo ejecutar y qué esperar	20
Championship	20
DatabaseTest	20
CityRepositoryTest y StadiumRepositoryTest	20

ChampionshipTest	20
JavaFX	20
29. Guía de lectura del código	20
30. Próxima evolución recomendada	21
Primero: tabla de posiciones	21
Segundo: eliminatorias y estadios	21
Tercero: ABM y GUI	21
Cuarto: reportes	21
31. Preguntas para defender el proyecto	21
32. Glosario breve	22

1. Qué sistema se está construyendo

El proyecto modela una Copa Internacional de Clubes. La información inicial de equipos, personas y países llega desde un archivo JSON; las ciudades y los estadios se leen desde PostgreSQL; el campeonato organiza zonas y partidos; un simulador produce resultados e incidencias; un presentador los muestra por terminal; y un repositorio binario permite guardar y recuperar el estado.

Fuente	Información	Componente que la transforma
torneo.json	Equipos, jugadores, DT, árbitros y países	TournamentLoader + DTO
PostgreSQL	Países relacionados, ciudades y estadios	DatabaseConnection + repositories
Memoria	Zonas, partidos, formaciones e incidencias	Championship + MatchSimulator
campeonato.dat	Estado serializado del torneo	ChampionshipRepository

IMPLEMENTADO	Flujo de grupos Carga JSON, validación, sorteo equilibrado, generación de 24 partidos, árbitros válidos, formaciones, resultados e incidencias.
PARCIAL	Infraestructura La lectura JDBC de ciudades y estadios funciona y Championship puede almacenarlos. El ABM todavía no existe.
PENDIENTE	Competencia completa Faltan tabla de posiciones, clasificados, eliminatorias, penales completos, estadios asignados a partidos y reportes.

El recorrido principal

```
torneo.json
-> Gson y DTO
-> TournamentLoader
-> TournamentValidator
-> TournamentData
-> Championship
  -> sorteo de zonas
  -> calendario de 24 partidos
  -> MatchSimulator
  -> MatchConsoleReporter
  -> ChampionshipRepository (campeonato.dat)
```

```
PostgreSQL Championship
-> DatabaseConnection
-> CityRepository
-> StadiumRepository
-> championship.loadVenues(cities, stadiums)
```

2. Organización de carpetas

Carpeta	Responsabilidad
Core/domain	Entidades y reglas básicas del problema: personas, equipos, partidos, formaciones, ciudades y estadios.
Core/dto	Forma intermedia que refleja la estructura del JSON.
Core/loader	Lectura, conversión y validación de los datos iniciales.
Core/simulation	Algoritmo que resuelve un partido y genera incidencias.
Core/incidents	Tipos de sucesos ocurridos durante un partido.
Core/console	Transforma un Match terminado en texto legible.
Core/persistence	Guarda y recupera Championship mediante serialización.
Core/Run	Puntos de entrada y coordinación del campeonato.
Infrastructure/database	Conexión JDBC y consultas a PostgreSQL.
GUI + Controller + resources	Inicio de interfaz JavaFX; todavía no opera el torneo.
src/test/java	Verificaciones automáticas del flujo de grupos.

Regla de dependencias

El dominio no debería depender de PostgreSQL, JavaFX ni la terminal. Una ciudad sabe qué país tiene, pero no sabe cómo fue leída. Un partido conoce sus incidencias, pero no sabe cómo se imprimen. Esta separación permite cambiar una interfaz o una base de datos sin reescribir las reglas deportivas.

- Domain: representa y protege el estado.
- Loader/Repository: obtiene datos externos y construye objetos.
- Simulation: aplica reglas para cambiar el estado de un partido.
- Console/GUI: presenta el estado sin decidir resultados.
- Run: conecta los componentes para ejecutar un caso de uso.

3. Configuración del proyecto

pom.xml

Maven describe el proyecto y sus bibliotecas. El código apunta a Java 24. Gson interpreta JSON, el driver PostgreSQL implementa JDBC y JavaFX aporta controles y carga FXML.

Dependencia	Versión	Uso
Gson	2.10.1	Deserializar torneo.json en DTO.
PostgreSQL JDBC	42.7.8	Abrir conexiones y ejecutar SQL.
JavaFX Controls	21.0.2	Botones, escenas y controles visuales.
JavaFX FXML	21.0.2	Construir la interfaz desde view.fxml.

Atención

Existe también `lib/gson-2.11.0.jar`. El proyecto mezcla una copia local 2.11.0 con la dependencia Maven 2.10.1. Conviene unificar la versión para evitar diferencias entre IntelliJ, terminal y Maven.

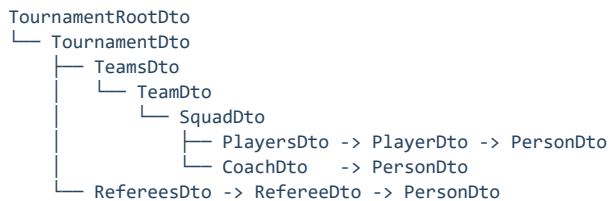
4. Entrada JSON y DTO

Por qué existen DTO y dominio

El JSON usa nombres y una estructura pensados para intercambio de datos. El dominio usa objetos con comportamiento. Los DTO son el puente: Gson los completa y TournamentLoader decide cómo convertirlos.

JSON	DTO	DOMINIO
"pais": "Argentina"	-> CountryDto{name}	-> Country("Argentina")
"posicion": "arquero"	-> PlayerDto.position	-> Position.GOALKEEPER
"persona": {...}	-> PersonDto	-> Player / Coach / Referee
"equipo": [...]	-> List<TeamDto>	-> List<Team>

Jerarquía de DTO



Adaptador de CountryDto

En el archivo, pais es un texto, no un objeto. Sin embargo, los DTO ahora usan CountryDto. TournamentLoader registra un adaptador que envuelve ese texto en un DTO.

```

private static final Gson GSON = new GsonBuilder()
    .registerTypeAdapter(
        CountryDto.class,
        (JsonDeserializer<CountryDto>)
            (json, type, context) -> new CountryDto(json.getAsString())
    )
    .create();

```

Idea central

Un DTO no decide reglas deportivas. Solo transporta datos. La validación y la creación de objetos compartidos ocurren después, en TournamentLoader y TournamentValidator.

5. TournamentLoader: del archivo a los objetos

Paso 1 - leer y deserializar

```

String json = Files.readString(Path.of(filePath));
TournamentRootDto root = GSON.fromJson(json, TournamentRootDto.class);

```

Files.readString obtiene todo el texto. Gson recorre la jerarquía de DTO basándose en los nombres de campos y en @SerializedName.

Paso 2 - catálogo único de países

```

Map<String, Country> countriesByName = new HashMap<>();

String key = name.trim().toLowerCase(Locale.ROOT);
return countriesByName.computeIfAbsent(
    key,
    ignored -> new Country(name)
);

```

El mapa evita crear varias instancias para el mismo nombre. Un equipo, un DT y un árbitro argentinos apuntan al mismo Country. La clave se normaliza para que las mayúsculas no creen duplicados.

Paso 3 - personas e IDs internos

El loader separa el identificador interno del número de documento. También divide el nombre completo del archivo, transforma la fecha con el formato dd/MM/yyyy y convierte las posiciones a Position.

Paso 4 - validación

```
TournamentData data = new TournamentData(teams, referees, countries);  
TournamentValidator.requireValid(data);  
return data;
```

No se entrega el objeto si el archivo incumple la estructura mínima. Así Championship nunca comienza con un plantel inválido.

6. Validación de datos

TournamentValidator revisa la coherencia global después de mapear el archivo.

- El torneo debe contener exactamente 16 equipos.
- Debe existir al menos un árbitro.
- Cada equipo debe tener 18 jugadores.
- Cada plantel debe incluir 2 arqueros, 6 defensores, 5 mediocampistas y 5 delanteros.
- Los errores se acumulan y se informan juntos mediante IllegalArgumentException.

Documentos repetidos

findDuplicateDocuments reúne personas por tipo y número de documento y devuelve una lista de colisiones. Es diagnóstico, no bloqueo automático, porque el archivo de origen ya contiene repeticiones. Antes de la entrega conviene decidir con la cátedra si esos números deben corregirse.

7. Modelo de personas y países

Person

Person es abstracta: concentra id, nombre, apellido, tipo y número de documento, y fecha de nacimiento. Player, Coach y Referee heredan esos datos. El constructor rechaza nombres vacíos, documentos no válidos y fechas futuras.

```
Person (abstracta, Serializable)
├── Player
├── Coach
└── Referee
```

Country

Country posee solo name, como se decidió para el proyecto. equals y hashCode comparan sin distinguir mayúsculas. También implementa Serializable para formar parte de campeonato.dat.

Player

- shirtNumber: número asignado por el loader.
- position: enum Position.
- characteristics: valores deportivos del JSON.
- statistics: estadísticas históricas del archivo, no estadísticas del campeonato actual.
- isSuspended: indica si debe perderse su próximo partido.

```
public double getAverageRating() {
    return characteristics.values().stream()
        .mapToInt(Integer::intValue)
        .average()
        .orElse(0);
}
```

serveSuspensionIfNeeded devuelve true si el jugador estaba suspendido y, a la vez, lo vuelve a habilitar. Lineup lo filtra, por lo que pierde exactamente esa convocatoria.

Coach y Referee

Coach agrega Country nationality y titlesWon. Referee agrega Country nationality y refereeYears. Los títulos influyen en la fuerza del equipo. La nacionalidad del árbitro determina si puede dirigir un encuentro.

8. Team y la fuerza competitiva

Team agrega jugadores de forma controlada, impide nulos y limita el plantel a 18. `getPlayers` devuelve una vista no modificable para que el exterior no altere la lista sin pasar por las reglas de Team.

Cálculo actual

```
double rankingScore = Math.max(0, 101 - ranking);
double coachScore = Math.min(100, coach.getTitlesWon() * 5.0);
```

```
strength = rankingScore * 0.35
          + averagePlayerRating * 0.55
          + coachScore * 0.10;
```

Factor	Peso	Interpretación
Ranking	35 %	Un ranking numéricamente menor produce más fuerza.
Plantel	55 %	Es el promedio de las características de los 18 jugadores.
DT	10 %	Cada título suma 5 hasta un máximo de 100.

Decisión de diseño

La fórmula no pertenece a la reglamentación real: es una regla creativa del equipo. Debe documentarse y poder explicarse. El componente aleatorio del simulador evita que esta fuerza determine siempre el ganador.

9. Championship: coordinador del torneo

Championship es la raíz del estado. Conserva países, equipos, árbitros, zonas, partidos, ciudades y estadios. Implementa `Serializable` para que todo el grafo pueda guardarse.

Construcción

```
TournamentData tournament = TournamentLoader.load(jsonPath);
teams      = new ArrayList<>(tournament.getTeams());
referees   = new ArrayList<>(tournament.getReferees());
countries  = new ArrayList<>(tournament.getCountries());
cities     = new ArrayList<>();
stadiums   = new ArrayList<>();
random     = randomReceived;
simulator  = new MatchSimulator(random);
zones      = drawBalancedZones();
matches    = generateGroupMatches(LocalDate.now());
```

Los datos JSON se cargan de inmediato. Las ciudades y estadios comienzan vacíos porque provienen de PostgreSQL y se incorporan después con `loadVenues`.

Dos formas de aleatoriedad

- `new Championship(path)`: usa una semilla variable; cada ejecución puede cambiar.
- `new Championship(path, 2026L)`: usa una semilla fija; sirve para pruebas reproducibles.

10. Sorteo equilibrado de zonas

Los 16 equipos se ordenan por ranking y se dividen en cuatro bombos de cuatro. Cada bombo se mezcla y entrega exactamente un equipo a cada zona.

Equipos ordenados por ranking

```
|— Bombo 1: índices 0..3
|— Bombo 2: índices 4..7
|— Bombo 3: índices 8..11
|— Bombo 4: índices 12..15
```

Cada zona recibe: 1 del Bombo 1 + 1 del Bombo 2
+ 1 del Bombo 3 + 1 del Bombo 4

```
for (int potIndex = 0; potIndex < 4; potIndex++) {
    List<Team> pot = new ArrayList<>()
        rankedTeams.subList(potIndex * 4, potIndex * 4 + 4)
    );
    Collections.shuffle(pot, random);
    for (int zoneIndex = 0; zoneIndex < 4; zoneIndex++) {
        zones.get(zoneIndex).addTeam(pot.get(zoneIndex));
    }
}
```

Esto cumple simultáneamente aleatoriedad y equilibrio. No pueden quedar cuatro equipos del primer bombo en una misma zona.

11. Generación de los 24 partidos

En cada zona de cuatro equipos se recorren pares sin repetir. El segundo índice comienza en homeIndex + 1, por eso no aparecen A vs A ni se duplica A vs B como B vs A.

```
for (int homeIndex = 0; homeIndex < 4; homeIndex++) {
    for (int awayIndex = homeIndex + 1; awayIndex < 4; awayIndex++) {
        // crear GroupMatch
    }
}
```

Pares por zona: 6
Zonas: 4
Total: 6 * 4 = 24 partidos

Se asigna una fecha consecutiva por partido. Las formaciones se pasan inicialmente como null porque todavía no fueron elegidas: MatchSimulator las crea al comenzar la simulación.

12. Regla de árbitros

Si los clubes son de países distintos, el árbitro no puede compartir nacionalidad con ninguno. Si ambos clubes son del mismo país, se aplica la excepción y cualquier árbitro resulta elegible.

```
if (home.getCountry().equals(away.getCountry())) {
    return true;
}
return !referee.getNationality().equals(home.getCountry())
    && !referee.getNationality().equals(away.getCountry());
```

chooseEligibleReferee filtra la lista, verifica que no esté vacía y elige una posición aleatoria.

13. Match y sus subtipos

Match es abstracta y reúne el estado común: fecha, local, visitante, árbitro, marcador, incidencias, dos Lineup y el indicador played.

```
Match (abstracta)
├─ GroupMatch
├─ FirstLegMatch
├─ SecondLegMatch
└─ FinalMatch
```

Estado inicial y estado final

Momento	homeLineup / awayLineup	goles	played
Al generar calendario	null	0 - 0	false
Al iniciar simulación	Lineup de 11	todavía calculándose	false
Al finalizar	Lineup conservada	resultado final	true

Subtipos eliminatorios

FirstLegMatch, SecondLegMatch y FinalMatch existen y son serializables. SecondLegMatch conserva goles de la ida; SecondLegMatch y FinalMatch tienen settledByPenalties. Aún no hay un servicio que genere cruces, calcule una serie, aplique goles de visitante o ejecute penales. Por eso estas clases son estructura parcial, no eliminatorias funcionales.

14. Lineup y FormationType

FormationType encapsula cantidades de defensores, mediocampistas y delanteros. Actualmente declara 4-3-3, 4-4-2, 3-5-2 y 4-5-1.

Selección

- Filtra jugadores suspendidos mediante serveSuspensionIfNeeded.
- Ordena los disponibles por valoración promedio, de mayor a menor.
- Selecciona un arquero y las cantidades solicitadas por FormationType.
- Si una suspensión impide completar una posición, rellena con los mejores jugadores restantes.
- Si ni siquiera hay once disponibles, lanza IllegalStateException.

```
available.stream()
    .filter(player -> !lineupList.contains(player))
    .limit(11 - lineupList.size())
    .forEach(lineupList::add);
```

Detalle importante

El relleno permite que el partido continúe aunque la formación solicitada no pueda cumplirse exactamente. La táctica pasa a ser una preferencia, no una garantía absoluta cuando hay suspendidos.

15. MatchSimulator paso a paso

1 - impedir doble ejecución

```
if (match.isPlayed()) throw new IllegalStateException(...);
```

2 - decidir tácticas

```
FormationType homeTactics = FormationType.FOUR_FOUR_TWO;
FormationType awayTactics = FormationType.THREE_FIVE_TWO;
Lineup homeLineup = new Lineup(match.getHomeTeam(), homeTactics);
Lineup awayLineup = new Lineup(match.getAwayTeam(), awayTactics);
```

Actualmente las tácticas están fijas: todos los locales usan 4-4-2 y todos los visitantes 3-5-2. FormationType permite evolucionar luego a una elección por DT o aleatoria.

3 - fuerza, localía y sorpresa

```
double homeStrength = home.getCompetitiveStrength() + 3.0;
double awayStrength = away.getCompetitiveStrength();
double surprise = random.nextGaussian() * 12.0;
```

El local recibe una ventaja de 3. El valor gaussiano puede mejorar o empeorar a cualquiera y produce resultados sorpresa.

4 - cantidad de goles

sampleGoals implementa un muestreo tipo Poisson: cuanto mayor es la expectativa, más probable resulta obtener varios goles. La expectativa se limita entre 0,25 y 3,5, y el ciclo limita el resultado práctico a seis.

```
homeExpected = 1.25 + (homeStrength - awayStrength + surprise) / 35.0;
awayExpected = 1.10 + (awayStrength - homeStrength - surprise) / 35.0;
```

5 - incidencias

- Cada gol recibe minuto aleatorio entre 1 y 90.
- Un gol tiene 12 % de probabilidad de ser penal.
- Si no es penal, tiene 3 % de probabilidad de ser en contra.
- Se generan de 0 a 3 amarillas por equipo.
- Dos amarillas al mismo jugador producen una roja un minuto después.
- Además existe 10 % de probabilidad de roja directa por equipo.
- Se intentan hasta tres sustituciones por equipo entre los minutos 55 y 90.

6 - cierre

```
match.setPlayed(true);
```

El motor calcula primero todo el encuentro y luego el presentador ordena los eventos. La salida parece una línea temporal, pero no hay un reloj que avance minuto a minuto.

16. Incidencias

```
Incident (abstracta: minute)
├─ Goal
├─ YellowCard
├─ RedCard
├─ Substitution
└─ PenaltyShootout
```

Clase	Datos	Estado
Goal	Minuto, autor, arquero, penal, gol en contra	Funcional
YellowCard	Minuto y jugador amonestado	Funcional
RedCard	Minuto y jugador expulsado	Funcional
Substitution	Minuto, jugador que entra y que sale	Funcional
PenaltyShootout	Solo minuto	Incompleta: faltan pateador y convertido/fallado

Orden y almacenamiento

Match guarda las incidencias en el orden en que el simulador las crea, no en orden cronológico.

MatchConsoleReporter copia la lista y la ordena por minute sin modificar el partido.

17. Salida por terminal

MatchConsoleReporter recibe un Match ya terminado. Imprime cabecera, fecha, árbitro, eventos y resultado. El patrón instanceof recupera los datos particulares de cada subtipo.

```
if (incident instanceof Goal goal) { ... }
if (incident instanceof YellowCard card) { ... }
if (incident instanceof RedCard card) { ... }
if (incident instanceof Substitution substitution) { ... }
```

Championship usa un Consumer<Match>. Después de simular cada partido, ejecuta afterEachMatch.accept(match). En main se pasa reporter::printMatch, una referencia al método de impresión.

```
championship.simulateGroupStage(reporter::printMatch);
```

Partidas ya terminadas

El main cuenta partidos pendientes. Si hay alguno, los simula. Si todos estaban jugados porque se recuperó campeonato.dat, imprime los resultados guardados en vez de terminar silenciosamente.

18. Persistencia por serialización

Core/persistence/ChampionshipRepository guarda el objeto Championship completo en campeonato.dat mediante ObjectOutputStream y lo recupera mediante ObjectInputStream.

```
try (ObjectOutputStream out =
    new ObjectOutputStream(new FileOutputStream(FILE_PATH))) {
    out.writeObject(championship);
}
```

Por qué tantas clases implementan Serializable

Al guardar Championship, Java recorre todas sus referencias: zonas, equipos, personas, partidos, formaciones, incidencias, ciudades y estadios. Cada objeto alcanzable debe ser serializable o estar marcado transient.

Random y MatchSimulator son transient

Esos objetos se excluyen del archivo. readResolve los reconstruye después de cargar para que el campeonato recuperado pueda seguir simulando.

```
@Serial
private Object readResolve() {
    this.random = new Random();
    this.matchSimulator = new MatchSimulator(this.random);
    return this;
}
```

Decisión del main

Si existe campeonato.dat, el usuario elige continuar o crear uno nuevo. La opción nueva reemplazará el archivo al guardar. La persistencia cumple parcialmente la consigna: conserva el estado, pero el flujo actual simula toda la fase de grupos de una vez, por lo que aún no se observa una pausa natural entre partidos.

19. PostgreSQL y JDBC

Modelo relacional observado

```
PAIS
  id BIGINT (PK)
  nombre
  |
  +-- CIUDAD
        id BIGINT (PK)
        nombre
        id_pais (FK -> pais.id)
        |
        +-- ESTADIO
              id BIGINT (PK)
              nombre
              ciudad (FK -> ciudad.id)
```

Esta normalización evita repetir el país en cada estadio. Java conserva la misma relación: Stadium -> City -> Country.

20. DatabaseConnection

La clase sigue el flujo JDBC enseñado: carga el driver, obtiene la contraseña desde una variable de entorno y abre Connection con DriverManager.

```
Class.forName("org.postgresql.Driver");

String password = System.getenv("CHAMPIONSHIP_DB_PASSWORD");

return DriverManager.getConnection(
    "jdbc:postgresql://localhost:5432/Championship",
    "postgres",
    password
);
```

Seguridad

La contraseña no se guarda en Git ni en el código. Cada configuración de ejecución de IntelliJ que use la base debe recibir CHAMPIONSHIP_DB_PASSWORD.

Responsabilidades

- DatabaseConnection conoce driver, URL y usuario.
- Los repositories conocen SQL.
- Championship conoce City y Stadium, pero no Connection ni ResultSet.

21. CityRepository

findAll recibe los países ya creados desde JSON. Ejecuta un JOIN entre ciudad y país, luego busca el Country correspondiente por nombre y crea City.

```
SELECT
    ciudad.id,
    ciudad.nombre AS ciudad,
    pais.nombre AS pais
FROM ciudad
JOIN pais ON ciudad.id_pais = pais.id
ORDER BY pais.nombre, ciudad.nombre

while (resultSet.next()) {
    long id = resultSet.getLong("id");
    String cityName = resultSet.getString("ciudad");
    String countryName = resultSet.getString("pais");
    Country country = findCountry(countries, countryName);
    cities.add(new City(id, cityName, country));
}
```

findCountry no crea un país nuevo. Si la base contiene un nombre inexistente en JSON, lanza IllegalArgumentException. Esto preserva el catálogo compartido, aunque exige que los nombres coincidan.

22. StadiumRepository e integración

StadiumRepository recibe las City ya cargadas. La consulta trae ciudad.id como ciudad_id; findCity encuentra el mismo objeto y construye Stadium.

```
SELECT
    estadio.id,
    estadio.nombre AS estadio,
    ciudad.id AS ciudad_id
FROM estadio
JOIN ciudad ON estadio.ciudad = ciudad.id
JOIN pais ON ciudad.id_pais = pais.id
ORDER BY pais.nombre, ciudad.nombre, estadio.nombre
```

Carga encadenada

```
Championship championship = new Championship("torneo.json");

List<City> cities = cityRepository.findAll(
    championship.getCountries()
);

List<Stadium> stadiums = stadiumRepository.findAll(cities);

championship.loadVenues(cities, stadiums);
```

Qué valida loadVenues

Rechaza listas nulas y verifica que cada Stadium referencie una City cuyo id exista en la lista recibida. Después copia ambas listas en Championship y expone vistas no modificables.

PARCIAL

Requisito de estadios

Los datos relacionales se leen y quedan integrados. Aún falta asignar estadios aleatoriamente a eliminatorias, controlar que no se repitan y realizar Alta/Baja/Modificación.

23. Clases de prueba

Clase	Qué comprueba
DatabaseTest	Abre Connection y muestra la base conectada.
CityRepositoryTest	Carga países del JSON y ciudades de PostgreSQL.
StadiumRepositoryTest	Carga ciudades, estadios y ejecuta loadVenues.
ChampionshipTest	Zonas, bombos, 24 partidos, árbitros, formaciones e incidencias.

Los tres tests JDBC están actualmente dentro de src/main/java. Para una entrega prolija deberían moverse a src/test/java y, más adelante, convertirse a JUnit. Por ahora son ejecutables manuales útiles para aislar errores.

24. JavaFX: qué existe y qué falta

AppLauncher delega en View. View carga /view.fxml, crea Scene y muestra Stage. view.fxml contiene un AnchorPane con dos botones vinculados por @FXML a Controller.controller.

```
AppLauncher.main
-> View.main
-> Application.launch
-> View.start(Stage)
-> FXMLLoader.load("/view.fxml")
-> Scene
-> stage.show()
```

PARCIAL

Interfaz

La ventana puede abrirse, pero los botones no tienen acciones, no existe navegación y no se muestran zonas, partidos, tablas, ciudades ni estadios.

Consigna de inglés

El código está mayormente en inglés, pero el título 'Gestión de Torneo', nombres de tablas SQL en español y algunos mensajes de consola mezclan idiomas. La consigna pide código e interfaces en inglés; conviene acordar con la cátedra si la base también debe renombrarse.

25. Programación Orientada a Objetos

Encapsulamiento

Los atributos son privados. Varias colecciones se exponen mediante Collections.unmodifiableList. Los métodos addPlayer, addTeam, setInitialLineups y loadVenues controlan cambios.

Herencia

Person, Match e Incident concentran datos comunes que reutilizan sus subclases.

Abstracción

Person, Match e Incident son conceptos incompletos por sí solos y se declaran abstractos.

Polimorfismo

Una List<Incident> almacena Goal, RedCard, YellowCard y Substitution; una List<Match> puede almacenar GroupMatch y futuros tipos eliminatorios. El polimorfismo de comportamiento todavía es débil: los subtipos de Match no sobrescriben simulate o determineWinner.

Composición

Team contiene Coach y Player; Stadium contiene City; City contiene Country; Match contiene Lineup e Incident. Estas relaciones modelan objetos formados por otros objetos.

Separación de responsabilidades

TournamentLoader carga, MatchSimulator simula, MatchConsoleReporter imprime, Championship coordina y los repositories consultan. Es una de las fortalezas actuales del diseño.

26. Correspondencia con la consigna

Requisito	Estado	Implementación actual
16 equipos y 4 zonas	Implementado	Validator + drawBalancedZones
Sorteo equilibrado	Implementado	Cuatro bombos mezclados
Planteles 2/6/5/5	Implementado	TournamentValidator
Partidos únicos de zona	Implementado	24 GroupMatch
Árbitro por nacionalidad	Implementado	isRefereeEligible
Simulación multifactor y aleatoria	Implementado	Team strength + Gaussian + Poisson
Formaciones	Implementado	Lineup + FormationType
Goles, cambios y expulsiones	Implementado	Incident subclasses + simulator
Suspensión de una fecha	Implementado	isSuspended + serveSuspensionIfNeeded
Tabla antes/después	Pendiente	No existe TeamStanding
Desempates y clasificados	Pendiente	No existe cálculo de posiciones
Cuartos/semis/final	Parcial	Clases creadas; flujo no implementado
Definición por penales	Parcial	Clase insuficiente; no hay algoritmo
Estadios desde BD	Parcial	Lectura e integración completas
Estadio aleatorio no repetido	Pendiente	Match no posee Stadium
ABM ciudad/estadio	Pendiente	Solo findAll
Persistencia del torneo	Implementado	Serialización campeonato.dat
Reportes y rankings	Pendiente	No hay servicios de reporte
JavaFX	Parcial	Ventana mínima sin operaciones

27. Riesgos y puntos a mejorar

- La estadística del mapa Player.statistics pertenece al archivo original; no se actualiza con los partidos simulados.
- Las tácticas local/visitante están fijas y no dependen del DT.
- Los incidentes pueden ser cronológicamente incoherentes entre sí porque se generan por grupos y luego solo se ordenan para imprimir.
- Una roja puede aparecer antes de un gol o cambio del mismo jugador porque no existe un motor minuto a minuto.
- Match no guarda Stadium, por lo que todavía no puede cumplir la regla de eliminatorias.
- Country se enlaza entre JSON y BD por nombre; diferencias de acento o escritura provocan error.
- ChampionshipRepository captura errores y devuelve null; para una GUI convendría propagar una excepción clara.
- La carpeta se llama persistence; en inglés técnico la forma habitual es persistence.
- Los DTO tienen campos públicos, aceptable para transporte, pero no para entidades de dominio.
- La GUI todavía está desacoplada por ausencia de funcionalidad, no porque exista una capa completa de casos de uso.

28. Cómo ejecutar y qué esperar

Championship

Ejecutar `Core.Run.Championship`. Si existe `campeonato.dat`, elegir 1 para continuar o 2 para crear un torneo nuevo. Un torneo nuevo simula y muestra 24 partidos. Uno ya completo vuelve a imprimir sus resultados guardados.

DatabaseTest

Ejecutar `Infrastructure.database.DatabaseTest` con `CHAMPIONSHIP_DB_PASSWORD` configurada. Debe mostrar conexión correcta y el catálogo `Championship`.

CityRepositoryTest y StadiumRepositoryTest

Necesitan la misma variable de entorno. El primero imprime ciudades; el segundo carga la cadena completa y muestra estadio, ciudad y país.

ChampionshipTest

Usa semilla fija, no necesita PostgreSQL y debe finalizar con 'All championship checks passed.'.

JavaFX

`GUI.AppLauncher` abre la ventana. No debe confundirse con `Championship`: ejecutar `JavaFX` no muestra todavía la simulación porque el controlador no la invoca.

29. Guía de lectura del código

Para comprenderlo sin saltos, estudiar en este orden:

- 1. `torneo.json` y los DTO: qué datos entran.
- 2. `TournamentLoader` y `TournamentValidator`: cómo se convierten y validan.
- 3. `Country`, `Person`, `Player`, `Coach`, `Referee` y `Team`: qué representa cada objeto.
- 4. `Championship.drawBalancedZones` y `generateGroupMatches`: cómo nace el calendario.
- 5. `FormationType` y `Lineup`: cómo se eligen once jugadores.
- 6. `MatchSimulator`: cómo se decide el resultado.
- 7. `Incident` y `MatchConsoleReporter`: cómo se guarda y presenta lo ocurrido.
- 8. `ChampionshipRepository`: cómo se conserva el estado.
- 9. `DatabaseConnection`, `CityRepository` y `StadiumRepository`: cómo entra PostgreSQL.
- 10. `GUI` y `Controller`: punto de partida de la futura interfaz.

30. Próxima evolución recomendada

Primero: tabla de posiciones

- Crear TeamStanding con puntos, PJ, PG, PE, PP, GF y GC.
- Actualizarla luego de cada GroupMatch.
- Ordenar por puntos, diferencia, goles a favor y resultado entre ambos.
- Mostrar la tabla antes y después de cada partido.
- Obtener dos clasificados por zona.

Segundo: eliminatorias y estadios

- Agregar Stadium a Match.
- Mantener una colección de estadios disponibles o utilizados.
- Crear cruces de cuartos con el orden indicado.
- Resolver ida, vuelta, diferencia, visitante y penales.
- Generar semifinales, final y campeón.

Tercero: ABM y GUI

- Agregar insert, update y delete a CityRepository y StadiumRepository.
- Dejar que PostgreSQL impida borrar ciudades con estadios.
- Crear pantallas JavaFX para administración y torneo.
- Conectar botones a casos de uso, no a SQL directo.

Cuarto: reportes

Una vez que el torneo acumule estadísticas confiables, construir goleadores, fair play, minutos, árbitros, cuadro de cruces e identificaciones PDF.

31. Preguntas para defender el proyecto

- ¿Por qué Country es un objeto compartido y no un String independiente?
- ¿Por qué un DTO no debería contener reglas deportivas?
- ¿Cómo garantiza el algoritmo un sorteo equilibrado y aleatorio?
- ¿Por qué se usa una semilla fija en pruebas?
- ¿Qué diferencia hay entre la estadística histórica del JSON y la del torneo actual?
- ¿Qué significa transient y por qué se reconstruye MatchSimulator?
- ¿Cómo evita try-with-resources conexiones abiertas?
- ¿Por qué StadiumRepository reutiliza City en lugar de crearla otra vez?
- ¿Qué requisito impide borrar una ciudad con estadios?
- ¿Qué partes prueban herencia, encapsulamiento, abstracción y polimorfismo?

32. Glosario breve

Término	Significado dentro del proyecto
DTO	Objeto temporal que refleja datos externos.
Dominio	Clases que representan las reglas y entidades del torneo.
Repository	Clase que recupera o guarda objetos en una fuente externa.
JDBC	API de Java para comunicarse con una base relacional.
ResultSet	Cursor sobre filas devueltas por un SELECT.
Serialización	Conversión de un grafo de objetos a bytes persistentes.
transient	Campo excluido de la serialización.
Semilla	Valor inicial que permite repetir una secuencia aleatoria.
Composición	Un objeto contiene y utiliza otros objetos.
Integridad referencial	Una clave foránea solo apunta a registros válidos.

Cierre

El proyecto ya posee un núcleo valioso: carga consistente, un modelo orientado a objetos, sorteo equilibrado, simulación con incidencias, persistencia y lectura JDBC. Para considerarlo completo según la consigna, la prioridad es transformar los resultados aislados en una competencia con tabla, clasificados, eliminatorias y estadísticas acumuladas.